Propuesta técnica del software

Proyecto formativo · GPS Guardián Escolar

Equipo del proyecto GPS Guardián Escolar

5 de noviembre de 2026

Versión documental: 2.5.7 · **Estado:** Estable

Tabla de contenido

- Resumen ejecutivo
    - Diagnóstico y problema
    - Objetivos
    - Alcance funcional y exclusiones
    - Solución tecnológica propuesta y situación adoptada
    - Alternativas valoradas
    - Seguridad, protección de datos y calidad
    - Estrategia de implementación y entrega
    - Estimación de recursos y sostenibilidad
    - Matriz de riesgos y mitigaciones
    - Criterios de aceptación y entregables
- Referencias y trazabilidad (APA 7.ª edición)

Resumen ejecutivo

Se propone —y se ha implementado en términos de código— una plataforma multiplataforma para supervisar trayectos escolares mediante registro de estudiantes, rutas, dispositivos vinculados, geoposicionamiento, zonas seguras y avisos. La solución utiliza un backend Spring Boot, cliente Expo y base PostgreSQL con migraciones Liquibase. La propuesta describe el producto observable, no promete disponibilidad continua ni exactitud GPS demostrada en campo.

1. Diagnóstico y problema

Los acudientes requieren información de trayecto que sea oportuna, restringida a responsables autorizados y suficientemente comprensible para reaccionar ante interrupciones de señal. Un sistema de ubicación escolar añade riesgos de privacidad de menores, errores de vinculación, caducidad de ubicaciones, dependencia de conectividad y costos por proveedores externos. Por ello el producto no se limita a mostrar un marcador en un mapa: necesita control de identidades, registro de estados, trazabilidad y procedimientos operativos.

Población usuaria: acudientes autorizados, estudiantes por medio de dispositivos vinculados, operadores/administradores y personal técnico. **Resultado perseguido:** facilitar la consulta controlada de trayectos y eventos escolares, sin sustituir procedimientos institucionales de seguridad ni servicios de emergencia.

2. Objetivos

General. Desplegar una solución cliente-servidor que permita administrar y observar trayectos de estudiantes mediante ubicación transmitida con autorización, con trazabilidad, seguridad de acceso y operación reproducible.

Específicos. (a) administrar cuentas y acceso de acudientes; (b) asociar estudiantes, rutas y dispositivos; (c) registrar coordenadas de trayecto en una BD versionada; (d) exponer seguimiento en tiempo real y notificaciones; (e) documentar instalación, límites, evidencia de pruebas y continuidad de operación.

3. Alcance funcional y exclusiones

Comprendido en el código consultado	Condicionado/no certificado
Auth, correo verificación, 2FA, JWT y sesiones	Entrega real de correo sin configurar proveedor
Estudiantes, guardianes, QR/códigos, rutas preferidas	Compatibilidad con todas las cámaras/teléfonos
Viajes, coordenadas, listados paginados, baja lógica	Precisión en todas las rutas y condiciones viales
STOMP/WebSocket, mapas, zonas seguras	Latencia garantizada o GPS background universal
Notificaciones y administración	Recepción push/SMS o operación Render certificada

No se incluye promesa de detección infalible de emergencias, vigilancia sin consentimiento, conducción automatizada, ni evidencia de APK publicado en esta revisión.

4. Solución tecnológica propuesta y situación adoptada

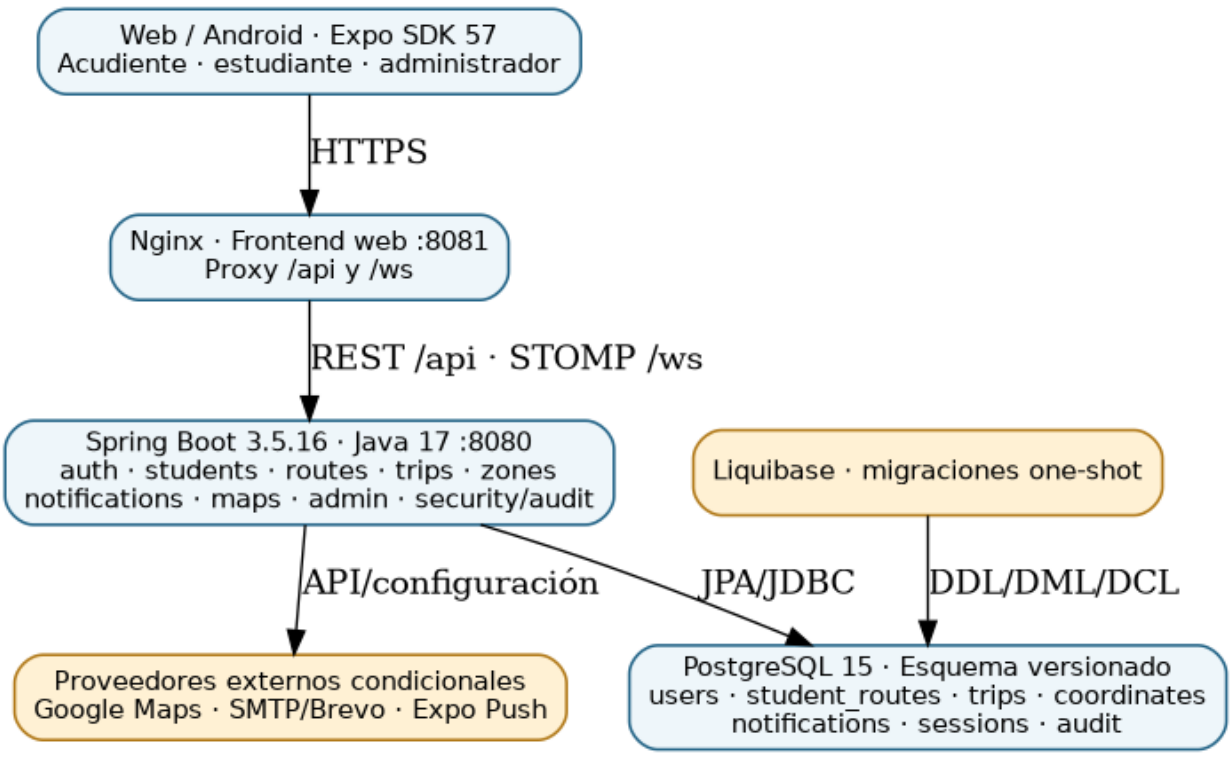


Figura 1. Arquitectura propuesta y reflejada en el código

Capa	Seleción y Justificacíon
Cliente	Expo SDK ~57 / React Native 0.86.3 / React 19.2.3; permite compartir lógica web/Android y usar cámara, ubicación y notificaciones.
Backend	Java 17 / Spring Boot 3.5.16, seguridad con JWT, REST, STOMP y Actuator; centraliza reglas y evita confiar en la UI.
Base	PostgreSQL 15, Liquibase; definición de esquema, índices y permisos fuera del JPA con ddl-auto: validate.
Integración	Maps como proxy autenticado con caché, rate limit y estrategias de resiliencia; correo mediante proveedor SMTP o Brevo; Expo Push sujeto a habilitación.
Infraestructura	Docker Compose encadena PostgreSQL, migración, API y frontend Nginx; despliegue público estudiado en Render.

Se trata de **un backend modular**, no una arquitectura de microservicios distribuidos. La separación por repositorio y por módulos no implica que cada módulo tenga proceso o base propia.

5. Alternativas valoradas

Opción	Ventaja	Consideración	Decisión documentada
Backend como servicio externo genérico	Menor código propio	Menor control del dominio y reglas de autorización	No adoptado como backend principal
Spring Boot + PostgreSQL	Control de reglas, datos y API	Requiere mantenimiento y despliegue	Adoptado
App 100 % nativa Android/iOS	Acceso directo a sensores	Duplicación de desarrollos	No adoptado en el código revisado
Expo multiplataforma	Reutilización de UI y servicios	Restricciones de GPS según plataforma/OEM	Adoptado
Backend por microservicio	Escalamiento independiente	Complejidad operativa mayor	No demostrado en este proyecto

6. Seguridad, protección de datos y calidad

Se exigen controles de autorización en backend, segmentación por estudiante autorizado, hash de tokens transitorios cuando aplique, expiración de sesiones, HTTPS público, secretos fuera de Git, datos mínimos de estudiantes y registros de auditoría. El operador deberá definir política de conservación de coordenadas, acceso a historial, respuesta a incidentes y retirada de dispositivos perdidos. La función SQL para depurar coordenadas de viajes cerrados requiere política y ejecución administrativa; su presencia **no prueba** que exista tarea automática de eliminación.

Requisitos no funcionales a validar: autenticación y revocación seguras, tiempos de respuesta medidos con carga definida, latencia de seguimiento con red variable, claridad en pérdida de señal, recuperación tras caída de proveedor, accesibilidad y usabilidad en móviles. No se asignan cifras obtenidas en pruebas que no se hayan corrido.

7. Estrategia de implementación y entrega

1. **Fijar release:** registrar commits aprobados y divergencias dev/develop–qa–main.
2. **Validar BD:** respaldo, Liquibase, comprobación de constraints/roles y migración de rutas preferidas.
3. **Instalar servicios:** entorno reproducible Docker con secretos de aplicación y bootstrap explícito.
4. **Ejecutar calidad:** mvnw clean verify, pruebas/lint del cliente, validación SQL, escenarios manuales y CI.
5. **Probar movilidad:** QR, permisos Android, GPS en segundo plano, WebSocket y notificaciones con dispositivos identificados.
6. **Promover ambiente:** QA y aprobación antes de Render/release; verificar endpoints y registrar evidencia.
7. **Operar y mejorar:** seguimiento de incidentes, respaldos, métricas, ajustes UX y control de versiones documentales.

8. Estimación de recursos y sostenibilidad

Los costos de alojamiento, PostgreSQL, tránsito, almacenamiento, Maps, correo, notificaciones, dominio y certificados dependen del plan contratado, consumo y proveedor en la fecha de contratación. No se presentan montos fijos sin cotización verificada. Elaborar una hoja de costos con columnas: concepto, volumen, tarifa vigente, estimación mensual, responsable y fecha de consulta. Agregar pruebas de carga para dimensionar CPU, RAM y conexiones a BD.

9. Matriz de riesgos y mitigaciones

Riesgo	Impacto	Tratamiento
GPS se suspende por el SO	Posición obsoleta / falsa sensación de certeza	Banner de última lectura, umbral 120s, prueba física
Claves expuestas	Acceso ilegítimo o factura externa	Secretos privados, scopes, rotación y auditoría
Divergencia de migraciones	API no inicia o datos inconsistentes	Liquibase y despliegue secuencial probado
Desfase cliente/API	Botones sin endpoint real	Inventario OpenAPI y pruebas de contrato
Menor vinculado incorrectamente	Riesgo de privacidad	Código controlado, confirmación y autorización servidor
Dependencia de Render/Maps/Push	Servicio degradado	Healthchecks, alertas, fallback comprensible, plan de contingencia

10. Criterios de aceptación y entregables

Técnicos: código aprobado y reproducible, migraciones verificadas, secretos no publicados, pruebas CI completadas, roles restringidos, endpoints de contrato alineados. **Funcionales:** registrar, vincular, definir ruta, iniciar viaje, consultar ubicación, reconocer pérdida de señal y desvincular, con evidencia positiva y negativa. **Documentales:** SRS, análisis, diseño, pruebas, procedimiento, acciones y manuales sincronizados con mismo corte de código. **Operativos:** backup restaurable y rollback ensayado, logs y responsable asignado.

Estado de aceptación a 2026-11-05: documento completo para revisión y PR; la aceptación del producto no está certificada por esta revisión documental.

Referencias y trazabilidad (APA 7.ª edición)

- GPS Guardian Escolar. (2026a). *Backend-Guardian-Escolar* [Código fuente, rama main, commit 1022058d]. GitHub. https://github.com/GPS-Guardian-Escolar/Backend-Guardian-Escolar/tree/1022058d545a9e7936edbff75950034ec4f98e88
- GPS Guardian Escolar. (2026b). *Fronted-GuardianEscolar-* [Código fuente, rama main, commit 3d9f59fa]. GitHub. https://github.com/GPS-Guardian-Escolar/Fronted-GuardianEscolar-/tree/3d9f59fa3e91b5ecc5b8ae86591187546b966668
- GPS Guardian Escolar. (2026c). *DB-Structure-Guardian-Escolar* [Migraciones SQL, rama main, commit 9278b1dc]. GitHub. https://github.com/GPS-Guardian-Escolar/DB-Structure-Guardian-Escolar/tree/9278b1dcb224301536d6158357f74daee74b20ba
- GPS Guardian Escolar. (2026d). *Docker-Infra-Guardian-Escolar* [Configuración de despliegue, rama main, commit 131e9895]. GitHub. https://github.com/GPS-Guardian-Escolar/Docker-Infra-Guardian-Escolar/tree/131e98954430e95e9108174eded9896b3f0d5272
- GPS Guardian Escolar. (2026e). *Docs-Guardian-Escolar: entregables 2.5.5 y suplementos 2.5.6* [Documentación del proyecto]. GitHub. https://github.com/GPS-Guardian-Escolar/Docs-Guardian-Escolar